

Reviewer Pack — Column-Subset Discrepancy Lower-Bounds Disjointness Protocol...

Entry 96f06d91d088 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-16 23:03:34 UTC

Column-Subset Discrepancy Lower-Bounds Disjointness Protocol Tree Leaves

Entry ID: 96f06d91d088 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-16 23:03:34 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial discrepancy theory (Chazelle-Matousek-style hereditary discrepancy of column subsystems, with Beck-Fiala / Banaszczyk-type bounds) **Field B** (complexity object): Deterministic two-party communication complexity of DISJOINTNESS measured by leaf count of the canonical protocol tree on random sub-instances

Statement:

For a random $\{0,1\}$ -matrix A in $\{0,1\}^{\{n \times n\}}$ with i.i.d. Bernoulli(1/2) entries (viewed as the communication matrix M_f of a DISJOINTNESS sub-instance restricted to row/column index sets X, Y of size n), let $\text{herdisc}_k(A)$ be the hereditary discrepancy $\max_{\{S \subseteq [n], |S|=k\}} \min_{\{\chi \in \{-1, +1\}^S\}} \|A_S \chi\|_\infty$, computed greedily by Beck-Fiala rounding on the k chosen columns. Let $L(A)$ be the number of leaves in the canonical DPLL-style deterministic protocol that, on inputs (x, y) in $\{0,1\}^n \times \{0,1\}^n$ with M_f indexed by A 's support, splits on the lowest-index coordinate where $x_i = y_i = 1$ (declaring NOT-DISJOINT) or otherwise descends until exhaustion (declaring DISJOINT). We conjecture: for every $n \geq 4$ and every k in $[2, \lfloor \log_2 n \rfloor + 2]$, $\log_2 L(A) \geq c \cdot k \cdot \text{herdisc}_k(A) / \sqrt{k \log n}$, with absolute constant $c \geq 1/8$, falsified by any single (A, k, seed) triple where the inequality is violated.

Rationale (proposer's reasoning):

Chazelle-Matousek discrepancy bounds the 'unavoidable irregularity' of any 0/1 incidence system, and Bourgain-Kashin-style column-subset dispersion is precisely what protocol-tree pruning must overcome: each leaf of the deterministic DPLL-style DISJOINTNESS protocol corresponds to a sign-balanced column restriction. Linking hereditary discrepancy of column subsystems to leaf-count is a dispersion-flavored angle that has not been pursued (the existing discrepancy/CC literature focuses on global $\text{disc}(M_f)$, not k -restricted herdisc). If true, this yields a dimension-free leaf lower bound bypassing rank-based methods.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 201e18fc436c5405

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds $\times$ 10 values of $n \in \{4, 6, 8, 10, 12, 16, 20, 24, 32, 40\} \times k \in [2, \text{floor}(\log_2 n) + 2]$ (k capped at 5), the inequality $\log_2(L(A)) \geq k \cdot \text{herdisc}_k(A) / (8 \cdot \sqrt{k \cdot \log n})$ must hold on $\geq 95\%$ of (n, k, seed) triples, with zero triples violating it by more than 10%.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - hereditary discrepancy communication complexity disjointness lower bound - Beck-Fiala rounding column subset protocol tree leaves - Banaszczyk discrepancy deterministic communication DPLL disjointness

Top relevant hits considered: - [<http://arxiv.org/abs/1505.03110v1>] Near-optimal bounds on bounded-round quantum communication complexity of disjointness - [<http://arxiv.org/abs/cs/0111062v2>] One-way communication complexity and the Neciporuk lower bound on formula size - [<http://arxiv.org/abs/cs/9908001v1>] A Lower Bound on the Average-Case Complexity of Shellsort - [<http://arxiv.org/abs/2508.01937v1>] An Improved Bound for the Beck-Fiala Conjecture - [<http://arxiv.org/abs/1306.6081v3>] An improvement of the Beck-Fiala theorem - [<http://arxiv.org/abs/2602.09948v2>] Non-Additive Discrepancy: Coverage Functions in a Beck-Fiala Setting - [<http://arxiv.org/abs/1102.4243v2>] Disjointness and unique ergodicity of C^* -dynamical systems - [<http://arxiv.org/abs/2409.00512v1>] Communicating in the Mediumband: What it is and Why it Matters

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def bernoulli(p):
        return 1 if random.random() < p else 0

    def herdisc_k(A, k):
        n = len(A)
        max_disc = 0
```

```

for S in itertools.combinations(range(n), k):
    disc = float('inf')
    for chi in itertools.product([-1, 1], repeat=k):
        A_S = [A[i][j] for i in range(n) for j in S if j == S.index(i)]
        norm = max(sum(A_S[j] * chi[j] for j in range(k)) for j in range(k))
        disc = min(disc, norm)
    max_disc = max(max_disc, disc)
return max_disc

def simulate_protocol_tree(A):
    n = len(A)
    leaves = set()

    def dfs(x, y, path):
        if all(x[i] != y[i] for i in range(n)):
            leaves.add(tuple(path))
            return
        for i in range(n):
            if x[i] == 1 and y[i] == 1:
                dfs(x[:i] + (0,) + x[i+1:], y[:i] + (0,) + y[i+1:], path + [i])
                dfs(x[:i] + (1,) + x[i+1:], y[:i] + (0,) + y[i+1:], path + [i])
                break

    for i in range(2**n):
        x = tuple((i >> j) & 1 for j in range(n))
        y = tuple((i >> j) & 1 for j in range(n, 2*n))
        dfs(x, y, [])

    return len(leaves)

n_values = [4, 6, 8, 10, 12, 16, 20, 24, 32, 40]
k_values = range(2, math.floor(math.log2(n)) + 3)
c = 1/8
total_leaves = 0
total_discs = 0
num_trials = 0

for n in n_values:
    for k in k_values:
        if k > 5:
            continue
        A = [[bernoulli(0.5) for _ in range(n)] for _ in range(n)]
        disc_k = herdisc_k(A, k)
        leaves = simulate_protocol_tree(A)
        total_leaves += leaves
        total_discs += disc_k * math.sqrt(k * math.log(n))
        num_trials += 1

mean_leaves = total_leaves / num_trials
std_dev = (sum((leaves - mean_leaves) ** 2 for leaves in [total_leaves]) / num_trials) ** 0.5
support_fraction = sum(1 for leaves, disc_k in zip([total_leaves], [total_discs]) if math.log2

return {
    "metric_name": "log2(L(A))",
    "metric_value": mean_leaves,
    "instances_tested": num_trials,

```

```

        "conjecture_holds": support_fraction >= 0.95,
        "counterexample": "" if support_fraction >= 0.95 else f"n={n_values[0]}, k={k_values[0]}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_leaves = sum(r["metric_value"] for r in results) / len(results)
    std_dev = (sum((r["metric_value"] - mean_leaves) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_leaves} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"n={n_values[0]}, k={k_values[0]}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e7a3676a.py", line 93, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e7a3676a.py", line 58, in run_trial
    k_values = range(2, math.floor(math.log2(n)) + 3)
                                   ^
UnboundLocalError: cannot access local variable 'n' where it is not associated with a value

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an UnboundLocalError before any trials produced data, so neither support nor falsification can be assessed. No RESULT line was emitted and the critic challenges on the basis of the crash. | next: Fix the scoping bug by defining n inside run_trial (or passing it as an argument) before computing k_values, then rerun the full 30-seed $\times$ 10-n grid.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	23565
2	preregistration	claude_max	opus	0	0	5814
3	novelty	claude_max	opus	0	0	3436
4	novelty	claude_max	opus	0	0	7642
5	test_gen	mistral	codestral-latest	0	0	311000
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	237209
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	295353
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	280498
9	judge	claude_max	opus	0	0	6418

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1170934 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 1, 'ollama_remote': 3}

(full prompt+response transcripts available in `research/audit/96f06d91d088.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/96f06d91d088.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/96f06d91d088.tar.gz` (if generated)